

step-by-step numerical example of *resource leveling with multiple resources*(Engineers and Machines). I'll keep it simple and show the trade-off between smoothing resource usage and project duration.

Example setup (inputs)

Resources available per day:

- **Engineers = 5**
- **Machines = 3**

Tasks (name, duration days, predecessors, Engineers required, Machines required):

Task	Dur (days)	Pred	Eng	Mac
A	3	—	3	1
B	4	A	2	2
C	2	A	4	0
D	5	B, C	1	1
E	3	C	2	1
F	2	D, E	2	1

1) Baseline (earliest start) schedule — no leveling

Compute earliest starts (ES) and finishes (EF) by forward pass:

Task	ES	EF
A	0	3
B	3	7
C	3	5
D	7	12
E	5	8

Task	ES	EF
F	12	14

Project duration (baseline) = 14 days.

Resource usage per day (baseline)

(Days indexed 0..13; each day shows Engineers, Machines, and active tasks)

Day	Eng used	Mac used	Active tasks
0–2	3	1	A
3–4	6	2	B, C ← <i>overallocated (Engineers>5)</i>
5–6	4	3	B, E
7	3	2	D, E
8–11	1	1	D
12–13	2	1	F

Problem: Days 3–4 Engineers = 6, but availability is 5 → overallocation.

2) Find leveling options

We compute task floats (how much each task can be delayed without increasing project end) — simplified results:

Task	ES	LS	Float (LS–ES)
A	0	11	11
B	3	10	7
C	3	12	9
D	7	9	2
E	5	11	6

Task	ES	LS	Float (LS–ES)
F	12	12	0 ← critical

Observation: C has a **large float (9)** — it can be shifted a lot without necessarily increasing finish, so C is a good candidate to delay to reduce the peak.

However, because **B and C** overlap during B’s whole execution (days 3–6), to eliminate overlap you must push **C** entirely out of B’s active window — that will **delay successor D** and may extend project duration. So there is a trade-off: **reduce peak resource use vs increase project finish**.

3) One leveled schedule (remove Engineer peak)

We choose to **delay C** so it does **not** overlap B at all. Place **C start = day 7** (well within C’s LS=12).

Recompute successors:

- B unchanged: ES=3 → EF=7
- C moved: ES=7 → EF=9
- D waits for both B and C → ES = max(7,9) = 9 → EF=14
- E (after C) → ES=9 → EF=12
- F after D & E → ES=14 → EF=16

New project duration = 16 days (extended by 2 days).

Leveled schedule tasks (selected fields)

Task	New ES	New EF
A	0	3
B	3	7
C	7	9
D	9	14
E	9	12
F	14	16

Resource usage per day (leveled)

Day	Eng used	Mac used	Active tasks
0–2	3	1	A
3–6	2	2	B
7–8	4	0	C
9–11	3	2	D, E
12–13	1	1	D
14–15	2	1	F

Result: Engineers never exceed 4 (≤ 5 capacity). Machines never exceed 2 (≤ 3 capacity). Peak smoothed — but project length increased from 14 → 16 days.

4) Summary — tradeoffs & rules of thumb

- Baseline schedule had **engineer overallocation** (peak 6 > capacity 5) on days 3–4.
- Leveling by delaying C removed the peak but **extended project duration by 2 days**.
- **General principle:** resource leveling usually smooths peaks but may increase project duration (unless you have slack you can use).
- When picking tasks to delay, prefer:
 - Tasks with **high float** (can be delayed without hurting finish), and
 - Tasks with resource demand that causes the peak.
- If you **must not** increase project finish, your choices are limited — you must only shift tasks within their floats and still respect capacity; sometimes the peak cannot be removed without extending finish.

5) Short checklist to perform leveling on any small project

1. Compute ES/EF and LS/LF → find floats.
2. Build day-by-day resource histogram for baseline.
3. Identify days where usage > availability (peaks).
4. For each peak, find candidate tasks active on that day with float > 0.
5. Delay candidate tasks (within float) to reduce peak; recheck successors and resource histogram.
6. Repeat until peaks solved or you decide project duration trade-off is acceptable.